Telecom Trigger Assignment (MSSQL)

Scenario

A telecom operator manages millions of subscribers, recharge transactions, internet packages, and customer activities daily using Microsoft SQL Server.

To improve:

- Security
- Audit tracking
- Data validation
- Customer activity monitoring
- Compliance reporting the telecom company wants to implement **Database Triggers**.

The database administrator is responsible for creating triggers that automatically execute during INSERT, UPDATE, and DELETE operations.

Students are required to design and implement triggers based on the following business requirements.

Database Tables

Students will use the following tables:

- subscribers
- recharge_transactions
- audit_logs
- package_history

Sample Database Setup

```
CREATE DATABASE TelecomTriggerDB;

GO


USE TelecomTriggerDB;

GO
```

```
-- Subscribers Table
CREATE TABLE subscribers (
    subscriber_id INT PRIMARY KEY IDENTITY(1,1),
    subscriber_name VARCHAR(100),
    phone_number VARCHAR(20),
    package_name VARCHAR(50),
    balance MONEY,
    created_at DATETIME DEFAULT GETDATE()
);

-- Recharge Transactions
CREATE TABLE recharge_transactions (
    recharge_id INT PRIMARY KEY IDENTITY(1,1),
    subscriber_id INT,
    recharge_amount MONEY,
    recharge_time DATETIME DEFAULT GETDATE()
);

-- Audit Logs
CREATE TABLE audit_logs (
    audit_id INT PRIMARY KEY IDENTITY(1,1),
    action_type VARCHAR(50),
    table_name VARCHAR(100),
    description VARCHAR(255),
    action_time DATETIME DEFAULT GETDATE()
);

-- Package History
CREATE TABLE package_history (
    history_id INT PRIMARY KEY IDENTITY(1,1),
    subscriber_id INT,
```

```
    old_package VARCHAR(50),
    new_package VARCHAR(50),
    changed_time DATETIME DEFAULT GETDATE()
);
GO
```

Sample Data

```
INSERT INTO subscribers
(subscriber_name, phone_number, package_name, balance)
VALUES
('Aung Aung', '091111111', 'Basic', 5000),
('Su Su', '092222222', 'Premium', 10000),
('Kyaw Kyaw', '093333333', 'Student', 3000);
GO
```

Assignment Questions

Question 1 — Audit New Subscriber Registration Requirement

Whenever a new subscriber is added into the subscribers table, automatically store an audit record into the audit_logs table.

Expected Information

- Action Type
- Table Name
- Subscriber Name
- Action Time

```sql
CREATE TRIGGER trg_AuditNewSubscriber
ON subscribers
AFTER INSERT
AS
BEGIN
    INSERT INTO audit_logs (action_type, table_name, description)
    SELECT 'INSERT', 'subscribers', 'New subscriber: ' + subscriber_name
    FROM inserted;
END;
GO

-- Test
INSERT INTO subscribers (subscriber_name, phone_number, package_name, balance)
VALUES ('Mya Mya', '094444444', 'Premium', 8000);
GO

SELECT * FROM audit_logs;
GO
```

100 % 4 0

Results Messages

	audit_id	action_type	table_name	description	action_time
1	1	INSERT	subscribers	New subscriber: Mya Mya	2026-06-05 09:50:50.623

Purpose: Automatically logs whenever a new subscriber is added.

Logic: The trigger captures the subscriber's name and inserts an audit record into audit_logs with action type INSERT.

Benefit: Provides audit tracking for new registrations.

Question 2 — Track Package Changes Requirement

Whenever a subscriber changes their internet package, automatically store:

- Old package
- New package
- Subscriber ID
- Changed time

into the package_history table.

```
78  CREATE TRIGGER trg_PackageChange
79  ON subscribers
80  AFTER UPDATE
81  AS
82  BEGIN
83      INSERT INTO package_history (subscriber_id, old_package, new_package)
84      SELECT d.subscriber_id, d.package_name, i.package_name
85      FROM deleted d
86      JOIN inserted i ON d.subscriber_id = i.subscriber_id
87      WHERE d.package_name <> i.package_name;
88  END;
89  GO
90
91  -- Test
92  UPDATE subscribers SET package_name = 'Gold' WHERE subscriber_id = 1;
93  GO
94
95  SELECT * FROM package_history;
96  GO
```

100 % 4 0

Results Messages

	history_id	subscriber_id	old_package	new_package	changed_time
1	1	1	Basic	Gold	2026-06-05 09:52:33.543

Purpose: Records changes in subscriber packages.

Logic: On UPDATE, the trigger compares old and new package values and inserts a record into package_history.

Benefit: Maintains a history of package upgrades/downgrades for compliance and reporting.

Question 3 — Log Deleted Subscribers Requirement

Whenever a subscriber record is deleted, automatically insert a log into audit_logs.

Expected Log

- Deleted subscriber name
- Delete action
- Delete time

```
101  CREATE TRIGGER trg_LogDeletedSubscriber
102  ON subscribers
103  AFTER DELETE
104  AS
105  BEGIN
106      INSERT INTO audit_logs (action_type, table_name, description)
107      SELECT 'DELETE', 'subscribers', 'Deleted subscriber: ' + subscriber_name
108      FROM deleted;
109  END;
110  GO
111
112  -- Test
113  DELETE FROM subscribers WHERE subscriber_id = 3;
114  GO
115
116  SELECT * FROM audit_logs;
117  GO
```

100 % 4 0

Results Messages

	audit_id	action_type	table_name	description	action_time
1	1	INSERT	subscribers	New subscriber: Mya Mya	2026-06-05 09:50:50.623
2	2	DELETE	subscribers	Deleted subscriber: Kyaw Kyaw	2026-06-05 09:56:17.563

Purpose: Logs subscriber deletions.

Logic: On DELETE, the trigger inserts a record into audit_logs with the subscriber's name and action type DELETE.

Benefit: Ensures accountability and traceability of deletions.

Question 4 — Validate Recharge Amount

Requirement

The telecom company does not allow recharge amounts smaller than:

1000 MMK

Create a trigger that prevents invalid recharge transactions.

```
122  CREATE TRIGGER trg_ValidateRecharge
123  ON recharge_transactions
124  AFTER INSERT, UPDATE
125  AS
126  BEGIN
127      IF EXISTS (SELECT * FROM inserted WHERE recharge_amount < 1000)
128      BEGIN
129          PRINT 'Recharge amount must be at least 1000 MMK';
130          ROLLBACK TRANSACTION;
131      END
132  END;
133  GO
134
135  -- Test
136  INSERT INTO recharge_transactions (subscriber_id, recharge_amount)
137  VALUES (1, 500); -- Should fail
138  GO
139
```

100 % 4 0

Messages

```
Recharge amount must be at least 1000 MMK
```

Purpose: Prevents recharge transactions below 1000 MMK.

Logic: The trigger checks the recharge_amount before insertion. If invalid, it raises an error and rolls back the transaction.

Benefit: Enforces business rules and prevents incorrect transactions.

Question 5 — Prevent Premium Subscriber Deletion Requirement

Subscribers using the Premium package are VIP customers.

The company policy does not allow deletion of Premium subscribers.

Create a trigger to prevent deletion.

```sql
CREATE TRIGGER trg_PreventPremiumDeletion
ON subscribers
INSTEAD OF DELETE
AS
BEGIN
    IF EXISTS (SELECT * FROM deleted WHERE package_name = 'Premium')
    BEGIN
        Print 'Cannot delete Premium subscribers';
        ROLLBACK TRANSACTION;
    END
    ELSE
    BEGIN
        DELETE FROM subscribers WHERE subscriber_id IN (SELECT subscriber_id FROM deleted);
    END
END;
GO

-- Test
DELETE FROM subscribers WHERE subscriber_id = 2; -- Should fail
GO
```

Purpose: Protects VIP customers from accidental deletion.

Logic: On DELETE, the trigger checks if the subscriber's package is Premium. If yes, deletion is blocked.

Benefit: Safeguards important customer records.

Question 6 — Combined INSERT and UPDATE Logging Requirement

Create one trigger that:

- Logs INSERT operations
- Logs UPDATE operations

for the subscribers table.

```
167  CREATE TRIGGER trg_InsertUpdateLogging
168  ON subscribers
169  AFTER INSERT, UPDATE
170  AS
171  BEGIN
172      INSERT INTO audit_logs (action_type, table_name, description)
173      SELECT
174          CASE WHEN d.subscriber_id IS NULL
175              THEN 'INSERT'
176              ELSE 'UPDATE'
177          END,
178          'subscribers',
179          'Subscriber change: ' + i.subscriber_name
180      FROM inserted i
181      LEFT JOIN deleted d ON i.subscriber_id = d.subscriber_id;
182  END;
183  GO
184
185  -- Test
186  UPDATE subscribers SET balance = balance + 1000 WHERE subscriber_id = 1;
187  GO
188
189  SELECT * FROM audit_logs;
190  GO
```

100 % 4 0

Results Messages

	audit_id	action_type	table_name	description	action_time
1	1	INSERT	subscribers	New subscriber: Mya Mya	2026-06-05 09:50:50.623
2	2	DELETE	subscribers	Deleted subscriber: Kyaw Kyaw	2026-06-05 09:56:17.563
3	3	UPDATE	subscribers	Subscriber change: Aung Aung	2026-06-05 10:07:10.473

Purpose: Logs both new registrations and updates in one trigger.

Logic: Distinguishes between INSERT and UPDATE operations and records them in audit_logs.

Benefit: Simplifies auditing by consolidating logging logic.

Question 7 — Prevent Duplicate Phone Numbers Requirement

Phone numbers must be unique in the telecom system.

Create a trigger to prevent duplicate phone number insertion.

```sql
CREATE TRIGGER trg_PreventDuplicatePhone
ON subscribers
AFTER INSERT
AS
BEGIN
    IF EXISTS (
        SELECT * FROM inserted i
        JOIN subscribers s ON i.phone_number = s.phone_number
    )
    BEGIN
        PRINT 'Duplicate phone number not allowed';
        ROLLBACK TRANSACTION;
    END
END;
GO

-- Test
INSERT INTO subscribers (subscriber_name, phone_number, package_name, balance)
VALUES ('Test User', '091111111', 'Basic', 2000); -- Should fail
GO
```

Purpose: Ensures phone numbers remain unique.

Logic: On INSERT, the trigger checks if the phone number already exists. If duplicate, it raises an error.

Benefit: Maintains data integrity and prevents conflicts.

Question 8 — Automatic Balance Update Requirement

Whenever a recharge transaction occurs:

- The recharge amount should automatically add to subscriber balance.

Example:

Before Balance Recharge After Balance

5000 2000 7000

```
SELECT subscriber_name, balance FROM subscribers WHERE subscriber_id = 1;
GO
```

100 % 4 0

Results Messages

	subscriber_name	balance
1	Aung Aung	6000.00

```
CREATE TRIGGER trg_AutoBalanceUpdate
ON recharge_transactions
AFTER INSERT
AS
BEGIN
    UPDATE subscribers
    SET balance = balance + i.recharge_amount
    FROM subscribers s
    JOIN inserted i ON s.subscriber_id = i.subscriber_id;
END;
GO

-- Test
INSERT INTO recharge_transactions (subscriber_id, recharge_amount)
VALUES (1, 2000); -- Balance should increase
GO

SELECT subscriber_name, balance FROM subscribers WHERE subscriber_id = 1;
GO
```

100 % 4 0

Results Messages

	subscriber_name	balance
1	Aung Aung	8000.00

Purpose: Updates subscriber balance automatically after recharge.

Logic: On INSERT into recharge_transactions, the trigger adds the recharge amount to the subscriber's balance.

Benefit: Eliminates manual updates and ensures accurate balances.

Question 9 — Recharge Audit Trigger Requirement

Whenever a recharge transaction occurs, automatically create an audit log.

```
242  CREATE TRIGGER trg_RechargeAudit
243  ON recharge_transactions
244  AFTER INSERT
245  AS
246  BEGIN
247      INSERT INTO audit_logs (action_type, table_name, description)
248      SELECT 'INSERT', 'recharge_transactions',
249             'Recharge of ' + CAST(i.recharge_amount AS VARCHAR) + ' for subscriber ' + CAST(i.subscriber_id AS VARCHAR)
250      FROM inserted i;
251  END;
252  GO
253
254  -- Test
255  INSERT INTO recharge_transactions (subscriber_id, recharge_amount)
256  VALUES (2, 3000);
257  GO
258
259  SELECT * FROM audit_logs;
260  GO
261
```

100 % 4 0

Results Messages

	audit_id	action_type	table_name	description	action_time
1	1	INSERT	subscribers	New subscriber: Mya Mya	2026-06-05 09:50:50.623
2	2	DELETE	subscribers	Deleted subscriber: Kyaw Kyaw	2026-06-05 09:56:17.563
3	3	UPDATE	subscribers	Subscriber change: Aung Aung	2026-06-05 10:07:10.473
4	6	UPDATE	subscribers	Subscriber change: Aung Aung	2026-06-05 10:14:43.537
5	7	UPDATE	subscribers	Subscriber change: Su Su	2026-06-05 10:15:41.163
6	8	INSERT	recharge_transactions	Recharge of 3000.00 for subscriber 2	2026-06-05 10:15:41.170

Purpose: Logs recharge transactions for auditing.

Logic: On INSERT into recharge_transactions, the trigger creates an entry in audit_logs with recharge details.

Benefit: Provides transparency and monitoring of financial transactions.

Question 10 — Security DDL Trigger Requirement

The telecom company does not allow developers to:

- DROP tables
- ALTER important tables

Create a DDL trigger to prevent schema modifications.

```
265  CREATE TRIGGER trg_PreventDDL
266  ON DATABASE
267  FOR DROP_TABLE, ALTER_TABLE
268  AS
269  BEGIN
270      PRINT 'DDL operations are not allowed on this database';
271      ROLLBACK TRANSACTION;
272  END;
273  GO
274
275  -- Test
276  DROP TABLE subscribers; -- Should fail
277  GO
```

Purpose: Prevents unauthorized schema changes.

Logic: A DDL trigger blocks DROP TABLE and ALTER TABLE operations.

Benefit: Protects database structure from accidental or malicious modifications.

Conclusion

This assignment demonstrates how triggers can:

- Automate auditing and compliance
- Enforce business rules
- Maintain data integrity
- Protect critical customer records
- Support financial accuracy

Together, these triggers enhance **security, monitoring, and reliability** in the telecom database.